

Prof. Tias Guns and Dr. Hendrik 'Henk' Bierlee



Partly based on slides from Pierre Flener, Guido Tack and Chris Beck.

Example (From: Steven S. Skiena's The Algorithm Design Manual)

Imagine you are a highly-in-demand actor, who has been presented with offers to star in n different movie projects under development. Each offer comes specified with the first and last day of filming. To take the job, you must commit to being available throughout this entire period. Thus, you cannot simultaneously accept two jobs whose intervals overlap.

For an artist such as yourself, the criteria for job acceptance is clear: you want to make as much money as possible. Because each of these films pays the same fee per film, this implies you seek the largest possible set of jobs (intervals) such that no two of them conflict with each other.

Title	Start	End
Tarjan of the Jungle	4	13
The Four Volume Problem	17	27
The President's Algorist	1	10
Steiner's Tree	12	18
Process Terminated	23	30
Halting State	9	16
Programming Challenges	19	25
Discrete Mathematics	2	7
Calculated Bets	26	31

How would you solve this problem?

Example (From: Steven S. Skiena's The Algorithm Design Manual)

Imagine you are a highly-in-demand actor, who has been presented with offers to star in n different movie projects under development. Each offer comes specified with the first and last day of filming. To take the job, you must commit to being available throughout this entire period. Thus, you cannot simultaneously accept two jobs whose intervals overlap.

For an artist such as yourself, the criteria for job acceptance is clear: you want to make as much money as possible. Because each of these films pays the same fee per film, this implies you seek the largest possible set of jobs (intervals) such that no two of them conflict with each other.

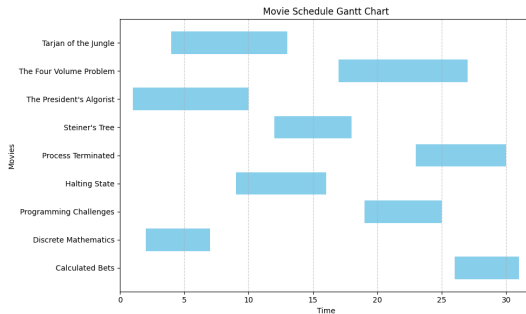
Title	Start	End
...	...	...

How would you solve this problem?

1. Trial and error on paper
2. Write a custom search algorithm
3. Reuse an existing, generic problem solving paradigm

1. Trial and error on paper

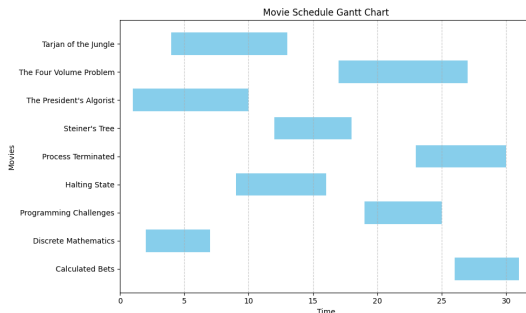
You would probably draw the start/end times to get an overview:



and then do some elimination and intuition-driven choices.

1. Trial and error on paper

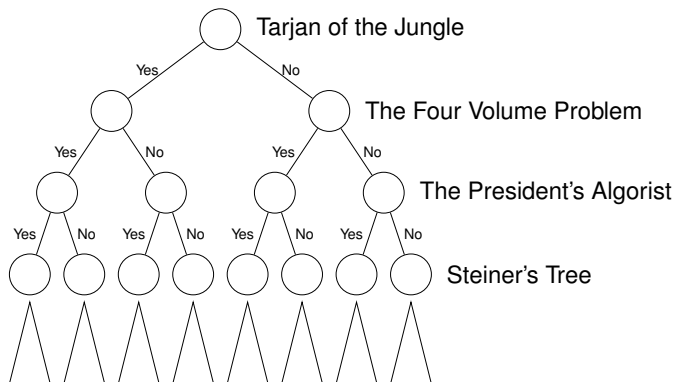
You would probably draw the start/end times to get an overview:



and then do some elimination and intuition-driven choices.

Now imagine every movie has a different fee, how to make sure you have the **optimal** solution?

2. Do exhaustive search with an algorithm



Example (From: Steven S. Skiena's The Algorithm Design Manual)

Imagine you are a highly-in-demand actor, who has been presented with offers to star in n different movie projects under development. Each offer comes specified with the first and last day of filming. To take the job, you must commit to being available throughout this entire period. Thus, you cannot simultaneously accept two jobs whose intervals overlap.

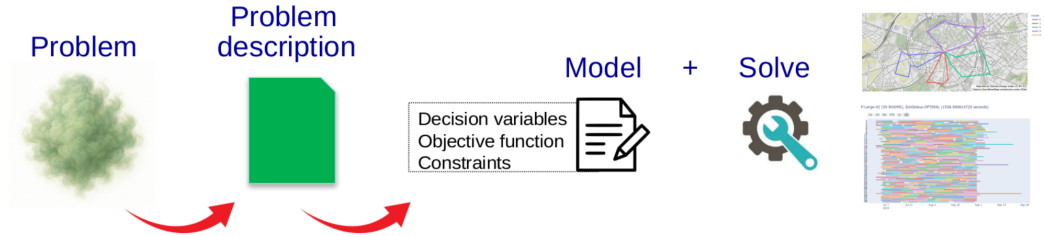
For an artist such as yourself, the criteria for job acceptance is clear: you want to make as much money as possible. Because each of these films pays the same fee per film, this implies you seek the largest possible set of jobs (intervals) such that no two of them conflict with each other.

Title	Start	End
...	...	...

How would you solve this problem?

1. Trial and error on paper
2. Write a custom search algorithm
3. Reuse an existing, generic problem solving paradigm $\Leftarrow$ **This Course**

Model + Solve paradigm



Outline

1. Combinatorial Optimisation Problems in the wild
2. Why are these problems hard?
3. What: Declarative Modelling
4. How: Solving
5. Course Information

Outline

1. Combinatorial Optimisation Problems in the wild
2. Why are these problems hard?
3. What: Declarative Modelling
4. How: Solving
5. Course Information

Example (Doctor rostering)

	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Doctor A							
Doctor B							
Doctor C							
Doctor D							
Doctor E							

Constraints to be satisfied:

1. #on-call doctors / day = 1
2. #operating doctors / weekday ≤ 2
3. #operating doctors / week ≥ 7
4. #appointed doctors / week ≥ 4
5. day off after operation day
6. ...

Objective function to be minimised: Cost: ...

Example (Doctor rostering)

	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Doctor A	call	none	oper	none	oper	none	none
Doctor B	appt	call	none	oper	none	none	call
Doctor C	oper	none	call	appt	appt	call	none
Doctor D	appt	oper	none	call	oper	none	none
Doctor E	oper	none	oper	none	call	none	none

Constraints to be satisfied:

1. #on-call doctors / day = 1
2. #operating doctors / weekday ≤ 2
3. #operating doctors / week ≥ 7
4. #appointed doctors / week ≥ 4
5. day off after operation day
6. ...



Objective function to be minimised: Cost: ...

Applications in Healthcare

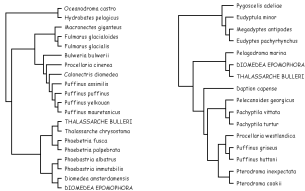
Doctor rostering



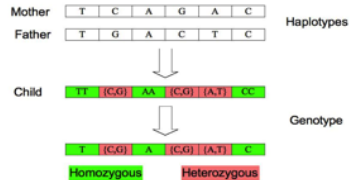
Radiotherapy beam optimisation



Phylogenetic supertree constr.



Haplotype inference



Example (Vehicle routing: parcel delivery)

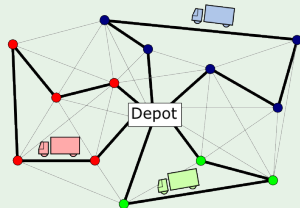
Given a depot with parcels for clients and a vehicle fleet,
find which vehicle visits which client when.

Constraints to be **satisfied**:

1. All parcels are delivered on time.
2. No vehicle is overloaded.
3. Driver regulations are respected.
4. ...

Objective function to be **minimised**:

- Cost: the total fuel consumption and driver salary.

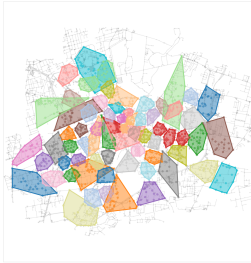


Example (Travelling salesperson: optimisation TSP)

Given a map and cities,
find a **shortest** route visiting each city once and returning to the starting city.

Applications in Transportation

Waste collection routing



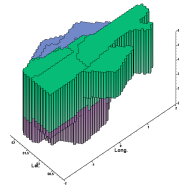
Container packing



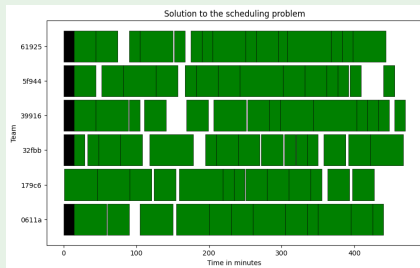
Contingency planning

Flow	Time Span	Hourly Rate
From: Arlanda	00:00 – 09:00	3
To: west, south	09:00 – 18:00	5
	18:00 – 24:00	2
From: Arlanda	00:00 – 12:00	4
To: east, north	12:00 – 24:00	3
...	...	...

Traffic control sectorisation



Example (Scheduling aircraft assembly transports (Airbus))



Constraints to be satisfied:

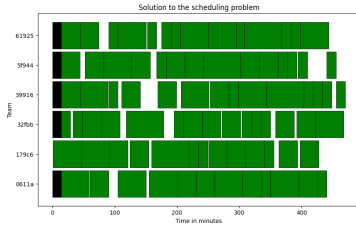
1. Each task has a pre-determined duration and allowed time-window
2. Each task can only be handled by a subset of worker teams
3. Some tasks have *precedence* relations (e.g. same aircraft)
4. ...

Objective function to be minimised:

Minimize number of teams needed, and the workload balance between teams.

Applications in Manufacturing

Transporting aircraft parts



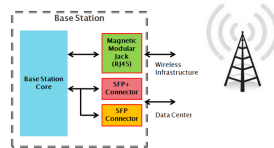
Robot programming



Timetabling

	Monday	Tuesday	Wednesday	Thursday	Friday
9:00	KMT200 Dynamics Differential Equations PS1	LARC 6007 Computer Graphics (S) Bust	KMT200 Numerical Analysis I Sitzman, G02		
10:00	KMT220 Dynamics Differential Equations MY01/Thursdays, 2,3	LARC 6007 Computer Graphics (S) Bust	Dynamics Differential Equations Sutton Engineering, Basement Theatre 5a	KMT220 Numerical Analysis I G02	KMT220 Dynamics Differential Equations MY01
11:00	CS0012 Algorithms and Data Structures		KMT212 Further Linear Algebra 1.5		KMT200 Dynamics Differential Equations Sutton Engineering, Basement Theatre 5a
12:00	KMT213 Further Linear Algebra Sutton, Theatre 5a	KMT200 Numerical Analysis I Sitzman, G02	CS017 Computer Graphics	KMT212 Further Linear Algebra Sutton, Theatre 5a	
1:00			PH45 Peer Assisted Study MATH 141B / 141T / 106L		KMT212 Further Linear Algebra Sutton Engineering, Basement Theatre 5a
2:00	CS0012 Computer Graphics 1.5			KMT212 Further Linear Algebra HQ17	
3:00		CS07 Tutorial			
4:00		CS012 Algorithms and Data Structures 1.5			

Base-station testing



Combinatorial optimization problems are everywhere!

Combinatorial = finding the best *combination* of objects from a finite set of possibilities.

We typically categorize problems by their **structural nature**, for example:

- ▶ **Assignment problems:** assign objects to fixed slots
(*doctor rostering, haplotype inference, base-station testing*)
- ▶ **Scheduling problems:** assign objects freely in time
(*transport scheduling, robot programming*)
- ▶ **Packing problems:** assign objects in 2D or 3D space
(*bin packing, container packing, carpet cutting*)
- ▶ **Routing problems:** enforce graph properties like cycles, connectivity
(*vehicle routing, waste collection, traffic control sectorisation*)

Outline

1. Combinatorial Optimisation Problems in the wild
- 2. Why are these problems hard?**
3. What: Declarative Modelling
4. How: Solving
5. Course Information

Let's reconsider:

Example (From: Steven S. Skiena's The Algorithm Design Manual)

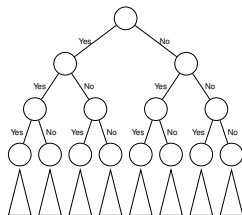
Imagine you are a highly-in-demand actor, who has been presented with offers to star in n different movie projects under development. Each offer comes specified with the first and last day of filming. To take the job, you must commit to being available throughout this entire period. Thus, you cannot simultaneously accept two jobs whose intervals overlap.

For an artist such as yourself, the criteria for job acceptance is clear: you want to make as much money as possible. Because each of these films pays the same fee per film, this implies you seek the largest possible set of jobs (intervals) such that no two of them conflict with each other.

Title	Start	End
...	...	...

Could you find a solution for 50 movies? For 1000s of movies across 100 actors?

Finding solutions with exhaustive search



- ▶ input = 50 actors and 1000 movies, is $2^{50 \times 1000}$ possible solutions...
- ▶ Search requires a worst-case **exponential** number of options to explore

Exponential: running time grows exponentially with input size n ,

$$T(n) \sim c^n \text{ for some constant } c > 1; \quad \text{big-oh notation: } \mathcal{O}(c^n)$$

Polynomial: running time is bounded by a polynomial function of input size n ,

$$T(n) \leq c * n^x \text{ for some constant } c > 0 \text{ and } x \geq 0; \quad \text{big-oh notation: } \mathcal{O}(n^x)$$

Does there exist a **polynomial** algorithm for combinatorial optimisation problems?

Limits of computation: $P \stackrel{?}{=} NP$ (Cook, 1971; Levin, 1973)

Informally:

- ▶ P = class of problems that need **no** search to be solved
 NP = class of problems that **might** need search to solve
- ▶ P = class of problems with easy-to-**compute** solutions (polynomial time)
 NP = class of problems with easy-to-**check** solutions (polynomial *check*)

Thus: Can search always be avoided ($P = NP$),
or is search sometimes necessary ($P \neq NP$)?

Limits of computation: $P \stackrel{?}{=} NP$ (Cook, 1971; Levin, 1973)

Informally:

- ▶ P = class of problems that need **no** search to be solved
 NP = class of problems that **might** need search to solve
- ▶ P = class of problems with easy-to-**compute** solutions (polynomial time)
 NP = class of problems with easy-to-**check** solutions (polynomial *check*)

Thus: Can search always be avoided ($P = NP$),
or is search sometimes necessary ($P \neq NP$)?

This is one of the seven **Millennium Prize** problems of the Clay Mathematics Institute (Massachusetts, USA), each worth 1 million US\$.

Problems that are solvable in polynomial time (in the input size) are considered **tractable**, aka **easy**.

Problems needing super-polynomial time are considered **intractable**, aka **hard**.

Examples of NP problems

Given a digraph (V, E) :

Examples

- ▶ Finding an **Euler tour** (which visits each *edge* once) takes $\mathcal{O}(E)$ time and is thus in P.
- ▶ Determining the existence of a **Hamiltonian cycle** (which visits each *vertex* once) is in NP (best known: $\mathcal{O}(1.657^n)$).

Examples

- ▶ Finding a **shortest path** takes $\mathcal{O}(V \cdot E)$ time and is thus in P.
- ▶ Determining the existence of a simple path (which has distinct vertices), from a given single source, that has *at least* a given number ℓ of edges is NP. Hence finding a **longest path** seems hard: increase ℓ starting from a trivial lower bound, until answer is 'no'.

More examples of NP problems

Examples

- ▶ **n -SAT**: Determining the satisfiability of a conjunction of disjunctions of n Boolean literals is in P for $n = 2$ but in NP for $n = 3$.
- ▶ **SAT**: Determining the satisfiability of a formula over Boolean literals is in NP.
- ▶ **Clique**: Determining the existence of a clique (complete subgraph) of a given size in a graph is in NP.
- ▶ **Vertex Cover**: Determining the existence of a vertex cover (a vertex subset with at least one endpoint for all edges) of a given size in a graph is in NP.
- ▶ **Subset Sum**: Determining the existence of a subset, of a given set, that has a given sum is in NP.

Search spaces are often larger than the number of particles in the universe!



Many important real-life problems are NP-hard or worse: their real-life instances can only be solved exactly and fast enough by **intelligent** search, unless $P = NP$.

NP-hardness is not where the fun ends, its where it begins!

Example (Optimisation TSP over n cities)

A brute-force algorithm evaluates all $n!$ candidate routes:

- ▶ A computer of today evaluates 10^6 routes / second:

n	time
11	40 seconds
14	1 day
18	203 years
20	77k years

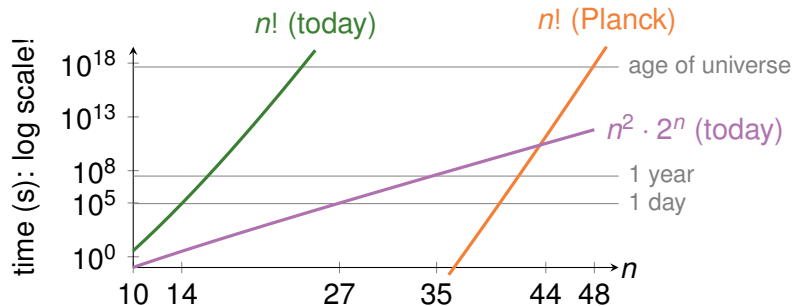
- ▶ Planck time is shortest useful interval: $\approx 5.4 \cdot 10^{-44}$ second;
a Planck computer would evaluate $1.8 \cdot 10^{43}$ routes / second:

n	time
37	0.7 seconds
41	20 days
48	$1.5 \cdot$ age of universe

The dynamic program by Bellman-Held-Karp “only” takes $\mathcal{O}(n^2 \cdot 2^n)$ time:
a computer of today takes a day for $n = 27$, a year for $n = 35$, the age of the universe for $n = 67$, and beats the $\mathcal{O}(n!)$ algo on Planck computer for $n \geq 44$.

Intelligent Search upon NP-Hardness

Do not give up but try to stay ahead of the curve:
there is an instance size until which an **exact** algorithm is fast enough!

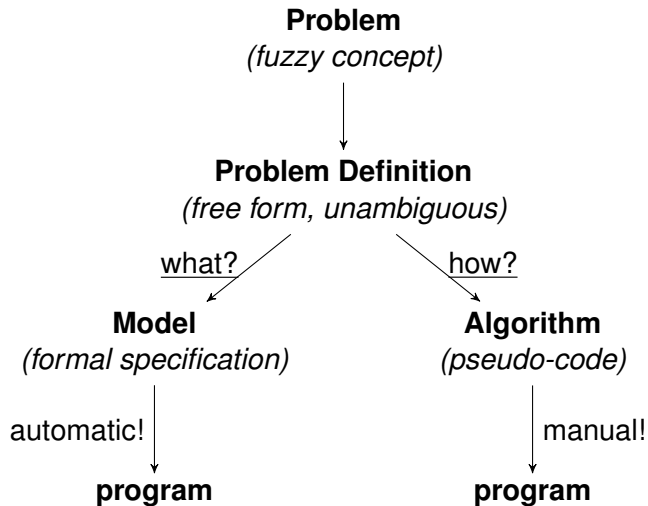


Concorde TSP Solver beats the **Bellman-Held-Karp** exact algo: it uses local search & approximation algos, but sometimes proves exactness of its optima. The largest instance solved exactly, in 136 CPU years in 2006, has $n = 85900$.

Outline

1. Combinatorial Optimisation Problems in the wild
2. Why are these problems hard?
- 3. What: Declarative Modelling**
4. How: Solving
5. Course Information

Modelling (declarative) vs Programming (imperative)



A **declarative problem solving paradigm** offers languages, methods, and tools for:

what: **Modelling** combinatorial problems in a **declarative** language.

and / or

how: **Solving** combinatorial problems **intelligently**:

- ▶ **Search**: Explore the space of possible assignments.
- ▶ **Inference**: Reduce the space to feasible (partial) assignments.

A **solver** is a program that takes a model and data as input and tries to solve that problem instance.

The ideas in this course extend to continuous optimisation, stochastic optimisation, planning and more

Examples (Declarative problem solving paradigms)

General-purpose solvers, taking model and data as input:

- ▶ SAT: Boolean satisfiability
- ▶ PB: Pseudo-Boolean Optimisation (0-1 linear constraints)
- ▶ SMT/OMT: SAT (resp. Optimisation) Modulo Theories
- ▶ MIP: Mixed Integer (Linear) Programming
- ▶ CP: Constraint programming
- ▶ ...

Examples (Non-declarative solving paradigms)

Methodologies (e.g. typically without separated concept of 'model' and 'solver'):

- ▶ Dynamic programming (DP)
- ▶ Greedy and Approximation algorithms
- ▶ Local search (LS)
- ▶ ...

Problem description



+



Definitions

A **combinatorial optimisation problem** consists of:

- ▶ **Decision variables**: the unknowns for which values have to be found
- ▶ **Domains**: for each variable what its allowed values are
- ▶ **Constraints**: relations between decision variables that must be satisfied
- ▶ optionally an **Objective function**: a mathematical function over the decision variables that must be minimized or maximized.

Definitions

An **assignment** maps each decision variable to a value within its domain; it is:

- ▶ **feasible** if all the constraints are satisfied;
- ▶ **optimal** if the objective function takes an optimal value.

The **search space** consists of all possible assignments.

A **solution** to a **satisfaction problem** is a feasible assignment.

An **optimal solution** to an **optimisation problem** is a feasible & optimal assignment.

Example (Sudoku)

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

8	1	2	7	5	3	6	4	9
9	4	3	6	8	2	1	7	5
6	7	5	4	9	1	2	8	3
1	5	4	2	3	7	8	9	6
3	6	9	8	4	5	7	2	1
2	8	7	1	6	9	5	3	4
5	2	1	9	7	4	3	6	8
4	3	8	5	2	6	9	1	7
7	9	6	3	1	8	4	5	2

The goal of Sudoku is to *complete* a 9x9 grid with numbers so that each row, column and 3x3 section contain each digit between 1 and 9 once.

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

8	1	2	7	5	3	6	4	9
9	4	3	6	8	2	1	7	5
6	7	5	4	9	1	2	8	3
1	5	4	2	3	7	8	9	6
3	6	9	8	4	5	7	2	1
2	8	7	1	6	9	5	3	4
5	2	1	9	7	4	3	6	8
4	3	8	5	2	6	9	1	7
7	9	6	3	1	8	4	5	2

Example (Sudoku CP model)

$G_{ij} \in \{1, 2, \dots, 9\},$

$\text{ALLDIFFERENT}(G_{i1}, G_{i2}, \dots, G_{i9}),$

$\text{ALLDIFFERENT}(G_{1j}, G_{2j}, \dots, G_{9j}),$

$\text{ALLDIFFERENT}(G_{p,q}, G_{p,q+1}, G_{p,q+2}),$

$G_{p+1,q}, G_{p+1,q+1}, G_{p+1,q+2},$

$G_{p+2,q}, G_{p+2,q+1}, G_{p+2,q+2}),$

$G_{ij} = \text{given}_{ij},$

$\forall i, j \in \{1, 2, \dots, 9\}$

$\forall i \in \{1, 2, \dots, 9\}$

$\forall j \in \{1, 2, \dots, 9\}$

$\forall p, q \in \{1, 4, 7\}$

if a value given_{ij} is given

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9					4		

8	1	2	7	5	3	6	4	9
9	4	3	6	8	2	1	7	5
6	7	5	4	9	1	2	8	3
1	5	4	2	3	7	8	9	6
3	6	9	8	4	5	7	2	1
2	8	7	1	6	9	5	3	4
5	2	1	9	7	4	3	6	8
4	3	8	5	2	6	9	1	7
7	9	6	3	1	8	4	5	2

Example (Sudoku in CPMpy (indexing offset 0))

```

1 import cpmypy as cp
2 #given = np.array(...) # load the clues, uses '0' for the empty cells
3 grid = cp.intvar(1,9, shape=given.shape, name="grid") # Decision variables
4 model = cp.Model(
5     [cp.AllDifferent(row) for row in grid],
6     [cp.AllDifferent(col) for col in grid.T], # numpy's Transpose
7     [cp.AllDifferent(grid[i:i+3, j:j+3]) \
8         for i in range(0, 9, 3) for j in range(0, 9, 3)],
9     grid[given!=0] == given[given!=0], # enforce the clues
10 )
11 model.solve()

```

8								
		3	6					
	7			9		2		
	5				7			
				4	5	7		
			1				3	
		1					6	8
		8	5				1	
	9				4			

8	1	2	7	5	3	6	4	9
9	4	3	6	8	2	1	7	5
6	7	5	4	9	1	2	8	3
1	5	4	2	3	7	8	9	6
3	6	9	8	4	5	7	2	1
2	8	7	1	6	9	5	3	4
5	2	1	9	7	4	3	6	8
4	3	8	5	2	6	9	1	7
7	9	6	3	1	8	4	5	2

Example (Sudoku in MiniZinc (indexing offset 1))

```

1 array[1..9,1..9] of 0..9: Given; % the hints, '0' for empty cells
2 array[1..9,1..9] of var 1..9: Grid;
3 constraint forall(row in 1..9) (all_different(Grid[row,...]));
4 constraint forall(col in 1..9) (all_different(Grid[..,col]));
5 constraint forall(i,j in {1,4,7}) (all_different(Grid[i..i+2,j..j+2]));
6 constraint forall(i in 1..9, j in 1..9 where Given[i,j] != 0)
   (Grid[i,j] = Given[i,j]); % enforce the hints
7 solve satisfy;

```

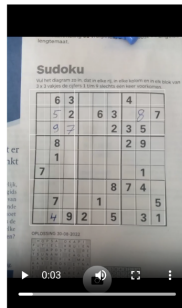
Fun with sudoku's

Sudoku Assistant: <https://visualsudoku.cs.kuleuven.be>



Sudoku Assistant

A.I. assistant combining machine learning and constraint solving techniques, to cleverly assist in solving pen-and-paper sudoku's.



HOW IT WORKS

Solving is instant and generating hints is real-time, both use CP solvers. The ML predictions (vision transformer) are the slowest.

Example (Agricultural experiment design)

	plot1	plot2	plot3	plot4	plot5	plot6	plot7
barley							
corn							
millet							
oats							
rye							
spelt							
wheat							

Constraints to be satisfied:

1. Equal sample size: Every grain is grown in 3 plots.
2. Equal growth load: Every plot grows 3 grains.
3. Balance: Every grain pair is grown in 1 common plot.

Instance: 7 plots, 7 grains, 3 grains/plot, 3 plots/grain, balance 1.

Parameterized Constraint Models

A model, e.g. a constraint specification, is typically written down as a parameterized constraint model.

- ▶ A **parameterized constraint model** has uninstantiated parameters. For example the number of grains and plots in the agricultural experiment design, or the given cells hints in a sudoku.
- ▶ The parameterized constraint model represents a *class* of problems.
- ▶ The uninstantiated parameters correspond to the input **data**.
- ▶ An **instance** is the combination of data and a parametrized constraint model, the instance is the result of instantiating the parameters, and in turn all the decision variables and constraints, with the data.

Example (Agricultural experiment design)

	plot1	plot2	plot3	plot4	plot5	plot6	plot7
barley	✓	✓	✓	—	—	—	—
corn	✓	—	—	✓	✓	—	—
millet	✓	—	—	—	—	✓	✓
oats	—	✓	—	✓	—	✓	—
rye	—	✓	—	—	✓	—	✓
spelt	—	—	✓	✓	—	—	✓
wheat	—	—	✓	—	✓	✓	—

Constraints to be satisfied:

1. Equal sample size: Every grain is grown in 3 plots.
2. Equal growth load: Every plot grows 3 grains.
3. Balance: Every grain pair is grown in 1 common plot.

Instance: 7 plots, 7 grains, 3 grains/plot, 3 plots/grain, balance 1.

General problem: **balanced incomplete block design (BIBD)**

In a BIBD, the plots are called **blocks** and the grains are called **varieties**:

Example (BIBD *integer* CP model)

$v = 7$, $b = 7$, Varieties, Blocks
 $k = 3$, $r = 3$, $\lambda = 1$, sample size, block size, balance

$B_{ij} \in \{0, 1\}$, $\forall i \in \{1, 2, \dots, v\}, \forall j \in \{1, 2, \dots, b\}$,

$\sum_{j=1}^b B_{ij} = k$, $\forall i \in \{1, 2, \dots, v\}$, every row must add up to sample size

$\sum_{i=1}^v B_{ij} = r$, $\forall j \in \{1, 2, \dots, b\}$, every columns must add up to block size

$\sum_{j=1}^b B_{ij} B_{i'j} = \lambda$, $\forall i, i' \in \{1, 2, \dots, v\}, i \neq i'$. every distinct row, scalar product = balance

In a BIBD, the plots are called **blocks** and the grains are called **varieties**:

Example (BIBD *integer* model in CPMpy)

```
1 varieties, blocks = 7, 7
2 sampleSize, blockSize, balance = 3, 3, 1
3
4 BIBD = cp.boolvar(shape=(varieties, blocks),name="matrix")
5
6 model = cp.Model(
7     # every row must add up to sampleSize
8     [cp.sum(row) == sampleSize for row in BIBD],
9     # every column must add up to blockSize
10    [cp.sum(col) == blockSize for col in BIBD.T],
11    # for every pair of distinct rows, the scalar product must sum up to balance
12    [cp.sum(row_i*row_j) == balance for row_i, row_j in all_pairs(BIBD)]
13 )
14
15 model.solve()
```

Consider the model fragment:

```
4      [cp.sum(row) == sampleSize for row in BIBD]
```

This constraint is **declarative**,
so read it using only the verb “must be” or similar:

*for each row row of $BIBD$,
the sum of values in row row
must be equal to $sampleSize$*

The constraint is **NOT procedural**:

*for each row row of $BIBD$,
we first sum the values in row row
and then check if that count equals $sampleSize$ -- !WRONG!*

The latter reading is appropriate for solution **checking**,
but solution **finding** performs no such procedural counting.

Symbolic model creation in Python

Declarative programming in a procedural language???

```
4      [cp.sum(row) == sampleSize for row in BIBD]
```

Yes, this is a declarative specification: the sum and comparison are not *executed*, instead they *create objects*!

The result is a list of CPMpy `Expression` objects.

These expressions are passed *symbolically* to a solver, who will create a search space and (procedurally) search for a solution to all constraints.

```
14      m = cp.Model([cp.sum(row) == sampleSize for row in BIBD])
15      print(m)
16      m.solve()
```

Example (BIBD *set-based* representation)

barley	{plot1, plot2, plot3}
corn	{plot1, plot4, plot5}
millet	{plot1, plot6, plot7}
oats	{plot2, plot4, plot6}
rye	{plot2, plot5, plot7}
spelt	{plot3, plot4, plot7}
wheat	{plot3, plot5, plot6}

Constraints to be **satisfied**:

1. Equal growth load: Every plot grows 3 grains.
2. Equal sample size: Every grain is grown in 3 plots.
3. Balance: Every grain pair is grown in 1 common plot.

Decision variables are a choice:

we could model the same problem with one *set* variable per grain variety.

Decision variables are a choice:

we could model the same problem with one *set* variable per grain variety.

Example (BIBD *set* CP model)

$v = 7, \quad b = 7,$ Varieties, Blocks
 $k = 3, \quad r = 3, \quad \lambda = 1,$ sample size, block size, balance

$\mathcal{B}_i \subseteq \{1, 2, \dots, b\},$ $\forall i \in \{1, 2, \dots, v\},$ Set of blocks for each variety

$\sum_{i=1}^v \mathbb{I}[j \in \mathcal{B}_i] = r,$ $\forall j \in \{1, 2, \dots, b\},$ Each block contains exactly block-size varieties

$|\mathcal{B}_i| = k,$ $\forall i \in \{1, 2, \dots, v\},$ Each variety appears in exactly sample-size blocks

$|\mathcal{B}_i \cap \mathcal{B}_j| = \lambda,$ $\forall i, j \in \{1, 2, \dots, v\}, i \neq j,$ balance between each pair of varieties

Notation: $\mathbb{I}[C]$ is the Iverson bracket: 1 if condition C holds, and 0 otherwise.

Note: not all modeling languages support *set* decision variables.

MiniZinc does, CPMpy does not.

Example (Doctor rostering)

	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Doctor A	call	none	oper	none	oper	none	none
Doctor B	appt	call	none	oper	none	none	call
Doctor C	oper	none	call	appt	appt	call	none
Doctor D	appt	oper	none	call	oper	none	none
Doctor E	oper	none	oper	none	call	none	none

Constraints to be satisfied:

1. #on-call doctors / day = 1
2. #operating doctors / weekday ≤ 2
3. #operating doctors / week ≥ 7
4. #appointed doctors / week ≥ 4
5. day off after operation day
6. ...



Objective function to be minimised: Cost: ...

Example (Doctor Shift Scheduling CP Model)

$R_{pd} \in \{\text{Appt}, \text{Call}, \text{Oper}, \text{Free}\},$

$\forall p \in \{1, \dots, \text{n_doctors}\}, d \in \{1, \dots, \text{n_days}\}$

1. #on-call doctors / day = 1: $\sum_{p=1}^{\text{n_doctors}} \llbracket R_{pd} = \text{Call} \rrbracket = 1,$

$\forall d \in \{1, \dots, \text{n_days}\}$

2. #operating doctors / weekday ≤ 2 : $\sum_{p=1}^{\text{n_doctors}} \llbracket R_{pd} = \text{Oper} \rrbracket \leq 2,$

$\forall d \in \{1, \dots, \text{n_days}\}, \text{ if } d \bmod 7 \in 1..5$

3. #operating doctors / week ≥ 7 : $\sum_{p=1}^{\text{n_doctors}} \sum_{d=s}^{s+6} \llbracket R_{pd} = \text{Oper} \rrbracket \geq 7,$

$\forall s \in \{1, \dots, \text{n_days}\}, \text{ if } s \bmod 7 = 1$

4. #appointed doctors / week ≥ 4 : $\sum_{p=1}^{\text{n_doctors}} \sum_{d=s}^{s+6} \llbracket R_{pd} = \text{Appt} \rrbracket \geq 4,$

$\forall s \in \{1, \dots, \text{n_days}\}, \text{ if } s \bmod 7 = 1$

5. day off after operation: $(R_{pd} = \text{Oper}) \rightarrow (R_{p,d+1} = \text{Free}),$

$\forall p \in \{1, \dots, \text{n_doctors}\}, d \in \{1, \dots, \text{n_days} - 1\}$

Notation: $\llbracket C \rrbracket$ is the Iverson bracket: 1 if condition C holds, and 0 otherwise.

Example (Data for our doctor rostering)

```
1 n_days = 7; n_doctors = 5
2 n_shifts = 4; Appt, Call, Oper, Free = range(n_shifts)
```

Example (CPMpy model for our doctor rostering (indexing offset 0))

```
1 roster = cp.intvar(0,n_shifts-1, shape=(n_doctors,n_days))
2 model = cp.Model(
3     # on-call/day = 1
4     [cp.Count(roster[:,d], Call) == 1 for d in range(n_days)],
5     # oper/weekday <= 2; assume d mod 7 == 0 for Monday, etc
6     [cp.Count(roster[:,d], Oper) <= 2 for d in range(n_days) if d % 7 <= 4],
7     # oper/week >= 7
8     [cp.Count(roster[:,s:s+7], Oper) >= 7 for s in range(0, n_days, 7)],
9     # appt/week >= 4
10    [cp.Count(roster[:,s:s+7], Appt) >= 4 for s in range(0, n_days, 7)],
11    # free after oper
12    [(roster[p,d] == Oper).implies(roster[p,d+1] == Free) for p in range(n_doctors
13    ) for d in range(n_days-1)],
14    # maximize nr of free shifts in weekend
15    model.maximize(cp.sum([cp.Count(roster[:,s+5:s+7], Free) for s in range(0, n_days,
16    7)]))
17 model.solve()
```

Example (Traveling Salesperson Problem)

$$\text{distance} = \begin{bmatrix} 0 & 85 & 162 & 231 \\ 85 & 0 & 98 & 128 \\ 162 & 98 & 0 & 146 \\ 231 & 128 & 146 & 0 \end{bmatrix}$$

$$n_cities = 4$$

$$\text{Next}_c \in \{0, \dots, n_cities - 1\}, \quad \forall c \in \{0, \dots, n_cities - 1\}$$

$$\text{minimize} \quad \sum_{c=0}^{n_cities-1} \text{distance}[c, \text{Next}_c]$$



Next:

AMS	BRU	LUX	CDG

How to ensure that *Next* represents a tour?

Example (Traveling Salesperson Problem)

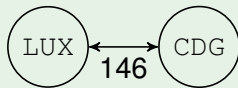
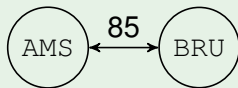
$$\text{distance} = \begin{bmatrix} 0 & 85 & 162 & 231 \\ 85 & 0 & 98 & 128 \\ 162 & 98 & 0 & 146 \\ 231 & 128 & 146 & 0 \end{bmatrix}$$

$$n_cities = 4$$

$$\text{Next}_c \in \{0, \dots, n_cities - 1\}, \quad \forall c \in \{0, \dots, n_cities - 1\}$$

$$\text{minimize} \quad \sum_{c=0}^{n_cities-1} \text{distance}[c, \text{Next}_c]$$

$$\text{ALLDIFFERENT}(\text{Next})$$



	AMS	BRU	LUX	CDG
Next:	BRU	AMS	CDG	LUX

So $\text{ALLDIFFERENT}(\text{Next})$ is too weak, it does not ensure *one* tour.

Example (Traveling Salesperson Problem)

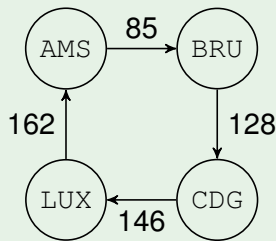
$$\text{distance} = \begin{bmatrix} 0 & 85 & 162 & 231 \\ 85 & 0 & 98 & 128 \\ 162 & 98 & 0 & 146 \\ 231 & 128 & 146 & 0 \end{bmatrix}$$

$n_cities = 4$

$Next_c \in \{0, \dots, n_cities - 1\}, \quad \forall c \in \{0, \dots, n_cities - 1\}$

minimize $\sum_{c=0}^{n_cities-1} \text{distance}[c, Next_c]$

CIRCUIT($Next$)

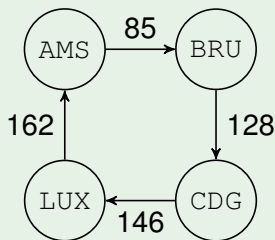


	AMS	BRU	LUX	CDG
Next:	BRU	CDG	AMS	LUX

For this, the CIRCUIT() global constraint is needed instead!

Example (Traveling Salesperson Problem (indexing offset 0))

```
1 n_cities = 4;
2 distance = cp.cpm_array([
3     [0,    85, 162, 231],    # Km from AMS
4     [85,    0,  98, 128],    # Km from BRU
5     [162,  98,   0, 146],    # Km from LUX
6     [231, 128, 146, 0  ]    # Km from CDG
7 ])
8 # Travel from c to Next[c]
9 Next = cp.intvar(0, n_cities-1, shape=
10     n_cities)
11 # Successor variables must form a circuit
12 model = cp.Model( cp.Circuit(Next) )
13
14 model.minimize(cp.sum(distance[c, Next[c]]
15     for c in range(n_cities)))
```



Next:

AMS	BRU	LUX	CDG
BRU	CDG	AMS	LUX

Special power of constraint programming languages:

Using a decision variable (`Next[c]`) as an index into an array (`distance[]`)

Decision Variables, Parameters, and Identifiers

- ▶ Parameters and decision variables in a model are concepts different from programming variables in an imperative or object-oriented program.
- ▶ A **parameter** in a model must be given a value, but only once: we say that the parameter is **instantiated**.
- ▶ A **decision variable** in a model is like a variable in mathematics: it is *not* given a value in a model, and its value is only fixed in a solution (if a solution exists).
- ▶ A parameter or decision variable is referred to by an **identifier** (a name like `Given`, `Grid`, `Next`, ...).

Modelling Concepts (end)

- ▶ A **constraint** is a restriction on the values that its decision variables can take together; equivalently, it is a Boolean-valued expression over decision variables that is asserted to be true.
- ▶ An **objective function** is a numeric expression over decision variables whose value is to be either minimised or maximised.
- ▶ Finally, we can ask a solver to **compute** different things:
 - ▶ find any satisfying solution
 - ▶ find all satisfying solutions
 - ▶ find an optimal solution
 - ▶ find all optimal solutions
 - ▶ count the number of satisfying/optimal solutions
 - ▶ prove that there is no solution
 - ▶ find a minimal subset of unsatisfiable constraints
 - ▶ ...

Correctness Is Not Enough for Models



Modelling is a craft!

- ▶ Different models of a problem may require different solve times, for the same solver on the same instance.
- ▶ Different models of a problem may scale differently for the same solver on instances of growing size.
- ▶ Different solvers may take different time for the same model on the same instance.

Good modellers are highly valued in industry!

Use solvers: based on decades of cutting-edge research, they are very hard to beat in finding optimal solutions.

Outline

1. Combinatorial Optimisation Problems in the wild
2. Why are these problems hard?
3. What: Declarative Modelling
- 4. How: Solving**
5. Course Information

Constraint-Based Modelling

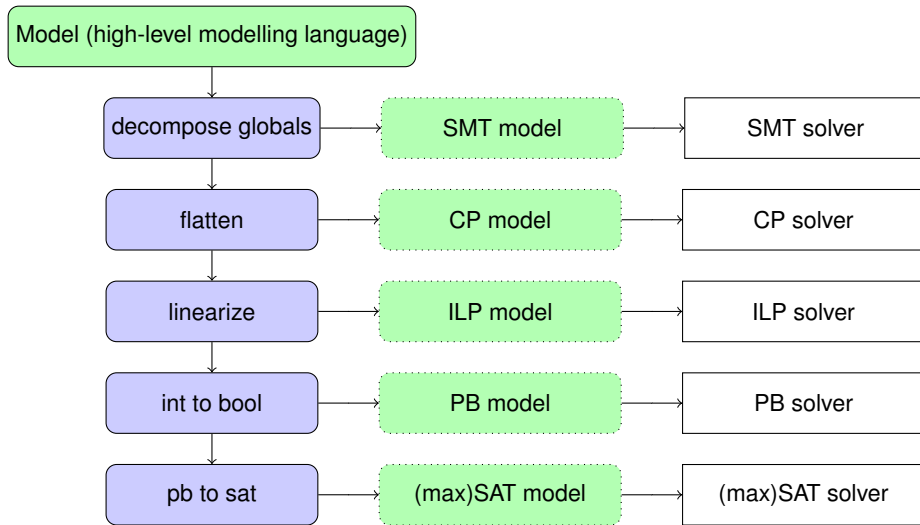
CPMpy is a Python-based constraint modelling *library* (not a solver):

- ▶ Decision variables are n-dimensional **numpy arrays**, and you can specify constraints using standard Python and NumPy functions.
only *Boolean* and *integer* decision variables are possible.
- ▶ Standard Python operators (+ - / & | `sum()` `abs()` `min()` `max()`) and comparisons (== >= > != <= <) can be used, and there is a large library of global constraints (`AllDifferent()`, `Circuit()`, `Cumulative()`, ...) and global functions (`Count()`, `Element()`, ...)
- ▶ There is support for both constraint satisfaction, optimisation (`solve()`) and solution counting/enumeration (`solveAll()`).

Compared to the library of a specific solver (e.g. *ortools* or *gurobi*), you can specify problems at a higher level, e.g. nested expressions, and use globals that the solver might not support.

CPMpy will *transform* this to a lower-level solver library for you.

From Model to Model to Solver: transformations



Solving a model

MiniZinc and CPMpy are **solver-independent** modelling frameworks: they can translate to multiple different solvers.

Expressiveness of key declarative solving paradigms:

- ▶ **SMT:** Bool&Int&String&... decision variables; logical, theory-specific constraints
- ▶ **CP:** Bool&Int decision variables; logical, mathematical, global constraints
- ▶ **MIP:** Integer(ILP)&Floating-point(MIP) decision variables; linear constraints & objective
- ▶ **PB:** Boolean decision variables; linear (Pseudo-Boolean) constraints & objective
- ▶ **SAT:** Boolean decision variables; clause constraints. MaxSAT: linear objective

There Are Many Solving Technologies

- ▶ Some technologies have **standardised** modelling languages across all solvers: SAT, PseudoBoolean, ILP/MIP, SMT
- ▶ Some technologies have **non-standardised** modelling languages across their solvers: CP and LCG (*although: FlatZinc, XCSP3*)
- ▶ Some technologies even have **no** modelling languages: local search, dynamic prog., and genetic algorithms are rather methodologies.

No technology universally dominates all the others

One should test several technologies on each problem!
(*easy in solver-independent modelling frameworks*)

How to Solve a Combinatorial Optimisation Problem?

1. **Model the problem**

2. **Have a solver solve it**

Easy, right?

How to Solve a Combinatorial Optimisation Problem?

1. Model the problem

- ▶ Understand the problem
- ▶ Choose the decision variables and their domains
- ▶ Formulate the constraints
- ▶ Formulate the objective function, if any
- ▶ Make sure the model really represents the problem; iterate

2. Have a solver solve it

- ▶ Choose a solving technology and solver
- ▶ Choose the hyper-parameters, potentially the search strategy
- ▶ Run the model and interpret the (lack of) solution(s)
- ▶ Debug or improve the model, if need be; iterate

Not so easy, but easier than implementing combinatorial algorithms from scratch!
(... where you would have to consider most modelling decisions either way)

Key take-aways, Model and Solve:

Advantages:

- + Declarative model of a combinatorial problem.
- + Easy adaptation to changing problem requirements.
- + Use of powerful solving technologies that are based on decades of cutting-edge research.

Disadvantages:

- Do I need to learn several modelling languages? **Not in this course!**
- Do I need to understand the used solving technologies in order to get the most out of them? **Yes, but ...!**

Outline

1. Combinatorial Optimisation Problems in the wild
2. Why are these problems hard?
3. What: Declarative Modelling
4. How: Solving
- 5. Course Information**

Lecture Plan

Modeling:

- ▶ L01: Introduction
- ▶ L02: Basic modeling
- ▶ L03: Global Constraints
- ▶ L04: Viewpoints and redundancy
- ▶ L05: Symmetry and dominance

Solving:

- ▶ L06: Solving technologies and encodings, part 1
- ▶ L07: Solving technologies and encodings, part 2
- ▶ L08: Portfolio solvers, algorithm configuration and parallelism

Advanced topics in CP research:

- ▶ L09: Topic TBA
- ▶ L10: Topic TBA